

UNINASSAU • FRONT-END FRAMEWORKS

JavaScript moderno III

Assincronismo e componentização sem React

Prof. Diogo Rodrigues

Pedido e entrega

1. Você pede

Faz o pedido no aplicativo e recebe um **comprovante**. A comida ainda não chegou.

2. Você espera

Continua sua vida. A cozinha trabalha em paralelo, você não fica parado na porta.

3. Chega (ou não)

A entrega chega, atrasa ou é cancelada. Cada final exige uma tela diferente.

comprovante = Promise • esperar = await • final = sucesso / vazio / erro

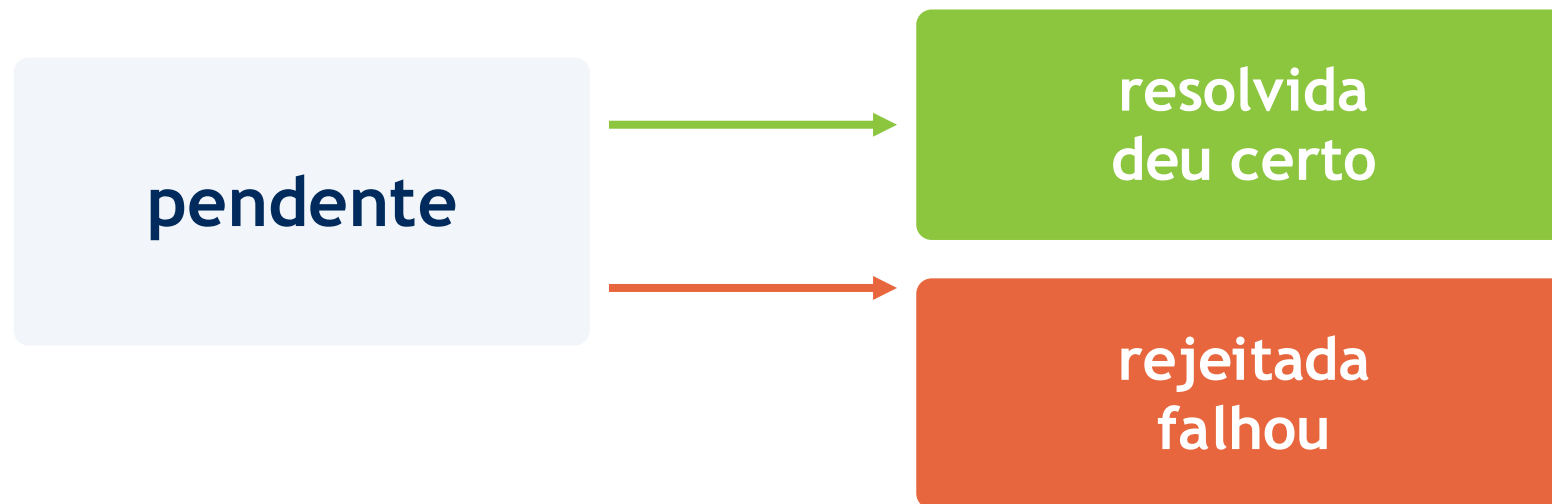
Em que ordem os textos aparecem?

```
console.log('A');
```

```
fetch('/api/itens').then(() => console.log('B'));
```

```
console.log('C');
```

Promise: o comprovante do pedido



Uma Promise é a promessa de um valor que ainda não existe.

Hoje nós apenas consumimos Promises prontas (fetch). Não vamos criar Promises na mão.

O código não espera sozinho

Síncrono

Uma linha por vez, em fila.
Cada linha só começa quando a anterior termina.
Exemplo: cálculos, montar texto.

Assíncrono

A linha é disparada e o programa segue.
O resultado volta depois.
Exemplo: fetch, temporizadores.

fetch devolve uma Promise

```
const p = fetch('/api/itens');  
console.log(p);    // Promise { <pending> } ← o comprovante  
  
// duas esperas: a resposta chegar, e o corpo virar objeto  
// 1) resposta HTTP      2) response.json()
```

Regra da aula: onde há espera, haverá await.

async / await: esperar sem travar

```
async function buscarItens() {  
  const resposta = await fetch('/api/itens');  
  const dados    = await resposta.json();  
  return dados;  
}  
  
// await só funciona dentro de uma função async
```

await = “segure aqui até a entrega chegar”

response.ok: erro que não é exceção

```
const resposta = await fetch(url);  
  
if (!resposta.ok) {  
    throw new Error('Falha ' + resposta.status);  
}  
  
const dados = await resposta.json();
```

404 e 500 NÃO fazem o fetch falhar. Sem esse if, você tenta desenhar uma página de erro como se fosse dado.

try / catch / finally

```
try {  
  const itens = await buscarItens();  
  renderizarLista(itens);  
} catch (erro) {  
  mostrarErro('Não foi possível carregar.');
```

```
} finally {  
  esconderCarregando();  
}
```

O usuário nunca deve ver uma tela em branco.

Quatro estados, quatro telas

Carregando

Pedido enviado.
Mostre esqueleto ou texto.

Sucesso

Lista com itens.
O caso feliz.

Vazio

Deu certo, mas veio 0 item.
Texto amigável, não erro.

Erro

Falhou.
Mensagem clara + tentar de novo.

Vazio ≠ Erro. Confundir os dois é o bug de interface mais comum de iniciante.

A API respondeu 500. O que o usuário vê?

```
const resposta = await fetch(url);  
const dados = await resposta.json();  
renderizarLista(dados);  
// sem if (!resposta.ok), sem try/catch
```

a) mensagem de erro b) tela em branco c) o carregando para sempre

Cada trio levanta 1, 2 ou 3 dedos ao mesmo tempo.

Componentização sem biblioteca

buscarItens()

Fala com a rede.
Devolve dados.
Não toca no HTML.

criarCartao(item)

Recebe 1 item.
Devolve 1 pedaço de
HTML.
Não sabe da lista.

renderizarLista()

Junta os cartões.
Escreve na tela.
Não busca nada.

iniciar()

Orquestra tudo.
Controla os estados.
Único ponto de
entrada.

dados → um cartão → lista na tela → quem manda

criarCartao(item)

```
function criarCartao(item) {  
  return `  
    <li class="cartao">  
      <h3>${item.nome}</h3>  
      <p>R$ ${item.preco}</p>  
    </li>  
  `;  
}
```

Mesma entrada, mesma saída, sempre. Fácil de testar.

renderizarLista(itens)

```
function renderizarLista(itens) {  
  if (itens.length === 0) {  
    lista.innerHTML = '<p>Nenhum item encontrado.</p>';  
    return;  
  }  
  lista.innerHTML = itens.map(criarCartao).join('');  
}
```

map devolve um ARRAY de textos. Sem join(""), aparecem vírgulas na tela.

iniciar()

```
async function iniciar() {  
  mostrarCarregando();  
  try {  
    const itens = await buscarItens();  
    renderizarLista(itens);  
  } catch (erro) {  
    mostrarErro('Não foi possível carregar.');  } finally {  
    esconderCarregando();  
  }  
}  
iniciar();
```

Três bugs escondidos. Encontrem e corrijam.

```
async function carregar() {  
  const resposta = await fetch('/api/itens');  
  const dados = resposta.json();  
  
  const ativos = dados.filter(item => item.ativo = true);  
  
  lista.innerHTML = ativos.map(criarCartao);  
}
```

Sintomas: “[object Promise]” na tela • todos os itens aparecem • vírgulas entre os cartões

Isso que fizemos já é um componente

Hoje, na mão

`criarCartao(item) → HTML`
`renderizarLista → innerHTML`
estados controlados por `if`
você decide quando redesenhar

No próximo encontro, com React

`<Cartao item={item} />`
`lista.map(...)` no JSX
estado com `useState`
React redesenha por você

Traga o desafio corrigido: vamos reescrever exatamente esta tela em React.